SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – ASSIGNMENT 1

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 1
Weightage:	15% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-2 & 3)		
Student Name:	K.Harsha Vardhan Reddy	**Reg. No.:**	**192425228**
Section / Batch:	**2024**	**Date:**	

Code of Conduct

I am K.Harsha Vardhan Reddy **(192425228)** *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Instructions

- Read each question carefully before answering. • Answer all questions in the space provided.
- Ensure your answers are accurate, clear, and neatly presented.
- Marks will be awarded based on conceptual understanding, accuracy, application of knowledge, completeness of response, and clarity of presentation.

	Understanding of the NLP concept (2)	Python implementation and logic (3)	Correctness of output (2)	Analysis and interpretation of results (2)	Code readability and documentation (1)	**TOTAL**
1						
2						
3						
4						
5						
6						
7						

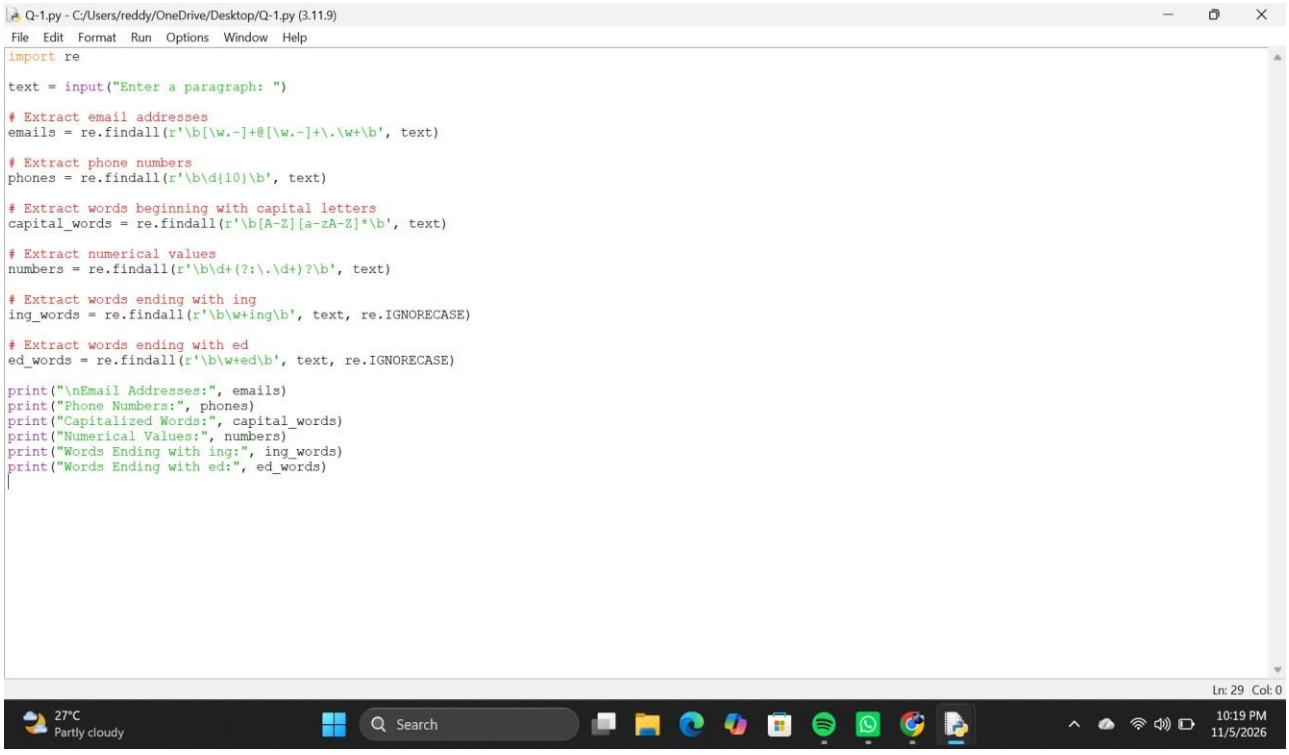
8						
9						
10						
					TOTAL MARKS	

Question 1: Regular Expression-Based Text Processing BTL 3 – Apply | 10 Marks

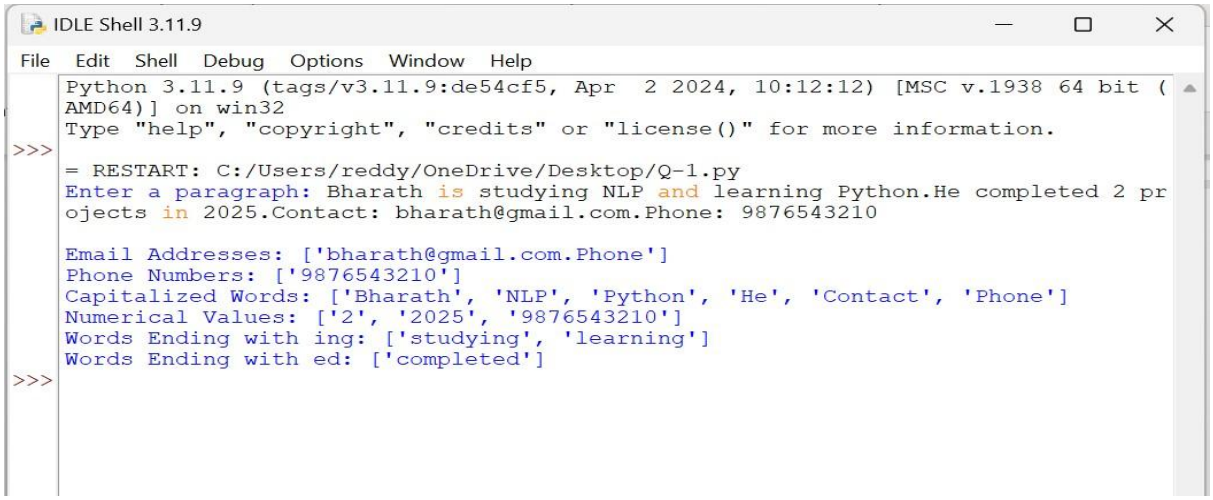
Develop a Python program using regular expressions to process a given paragraph and identify the following language features:

1. Extract all email addresses and phone numbers.
2. Identify all words beginning with a capital letter.
3. Extract all numerical values from the text.
4. Identify words ending with ing and ed.
5. Display the extracted results separately.

CODE:



OUTPUT:

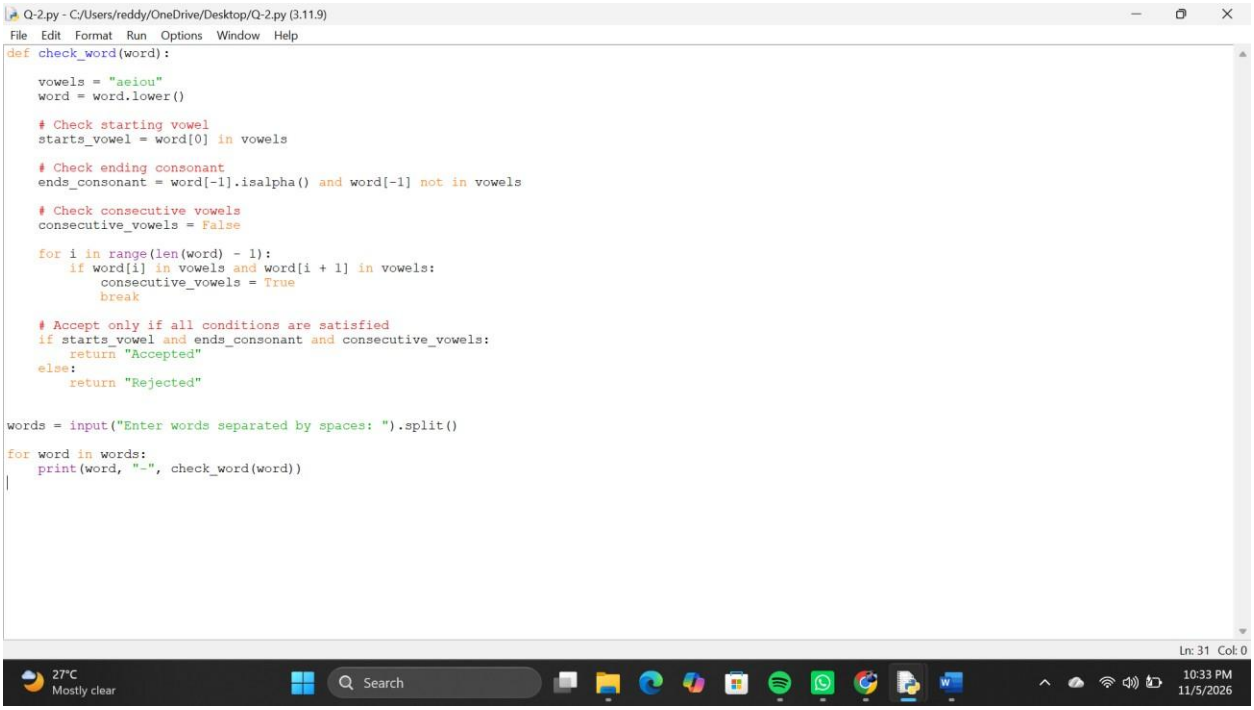


Question 2: Finite State Automata for Word Pattern Recognition BTL 3 – Apply | 10 Marks

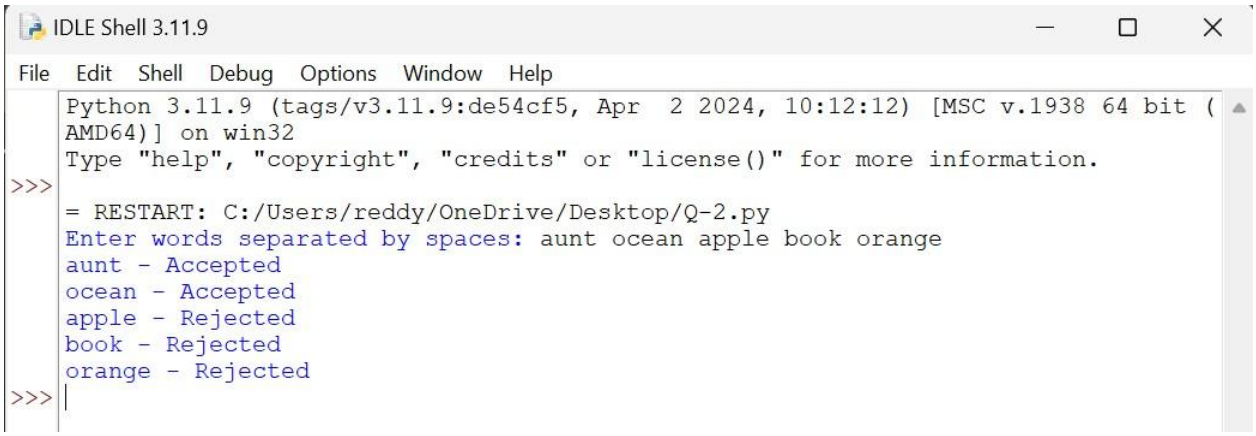
Implement a Finite State Automaton (FSA) using Python to recognize valid English word patterns based on the following conditions:

1. Accept words starting with a vowel.
2. Identify words ending with a consonant.
3. Recognize words containing consecutive vowels.
4. Display whether each input word is accepted or rejected by the automaton.

CODE:



OUTPUT:

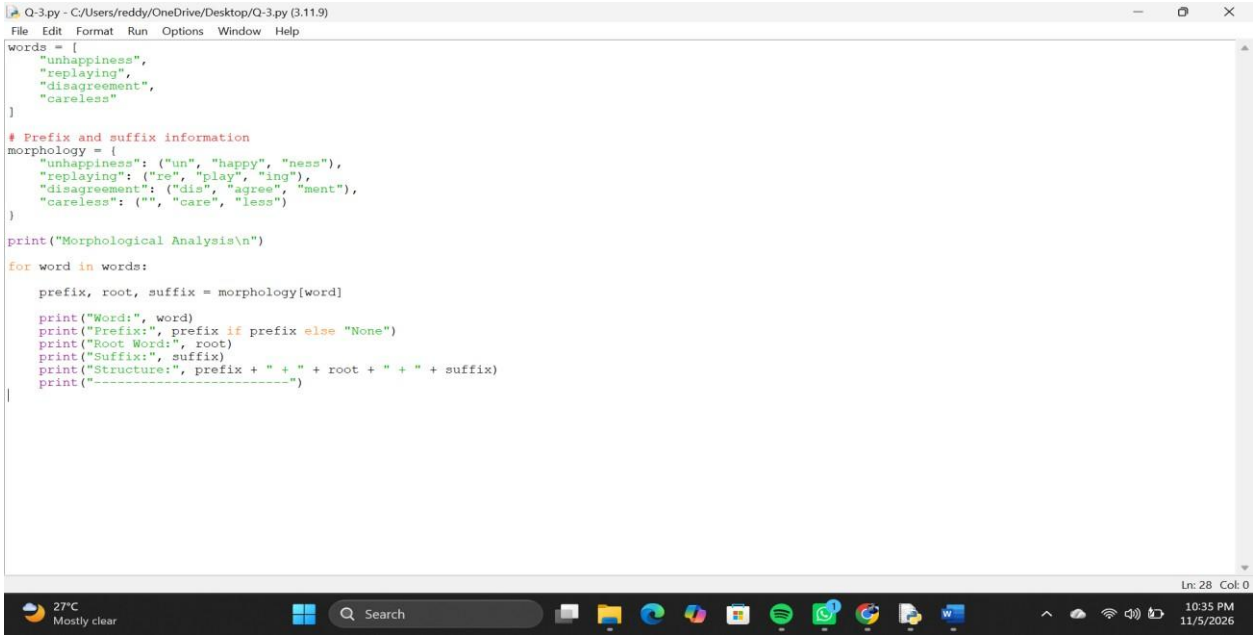

IDLE Shell 3.11.9
File Edit Shell Debug Options Window Help
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/reddy/OneDrive/Desktop/Q-2.py
Enter words separated by spaces: aunt ocean apple book orange
aunt - Accepted
ocean - Accepted
apple - Rejected
book - Rejected
orange - Rejected
>>>

BTL

Question 3: Morphological Analysis of English Words 4 – Analyze | 10 Marks Develop a Python program to perform morphological analysis on a given set of English words.

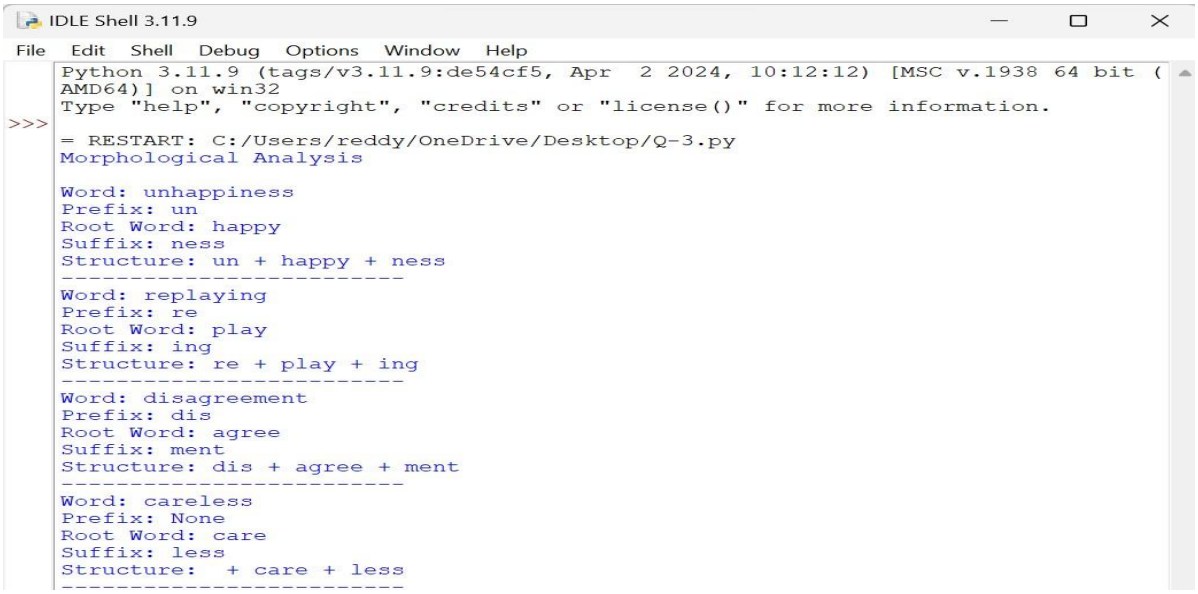
1. Identify the root word and its affixes.
2. Separate prefixes and suffixes wherever applicable.
3. Identify the morphological structure of each word.
4. Analyze words such as unhappiness, replaying, disagreement, and careless.
5. Display the morphological components in a structured format.

CODE:



OUTPUT:

BTL



Question 4: Inflectional and Derivational Morphology 3

– Apply | 10 Marks Write a Python program to identify and classify inflectional and derivational word forms from a given text.

1. Identify plural noun forms.
2. Identify past-tense and present participle verb forms.
3. Identify words formed using prefixes and suffixes.
4. Classify the identified words into inflectional and derivational categories.
5. Display the original word and its morphological classification.

CODE:

BTL

```
Q-4.py - C:/Users/reddy/OneDrive/Desktop/Q-4.py (3.11.9)
File  Edit  Format  Run  Options  Window  Help
text = input("Enter a sentence: ")

words = text.lower().split()

# Example inflectional and derivational suffixes
inflectional_suffixes = ("s", "es", "ies", "ed", "ing")
derivational_suffixes = ("ness", "ment", "ful", "less", "ly")

prefixes = ("un", "re", "dis", "pre")

print("\nMorphological Classification\n")

for word in words:

    word = word.strip(".,!?")

    if word in ("is", "was", "has", "his", "this"):
        continue

    if word.endswith(derivational_suffixes) or word.startswith(prefixes):
        print(word, "- Derivational")

    elif word.endswith(inflectional_suffixes):
        print(word, "- Inflectional")
```

Ln: 25 Col: 0

27°C Mostly clear | Search | 10:38 PM 11/5/2026

OUTPUT:

```
IDLE Shell 3.11.9
File  Edit  Shell  Debug  Options  Window  Help
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr  2 2024, 10:12:12) [MSC v.1938 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/reddy/OneDrive/Desktop/Q-4.py
Enter a sentence: The students are playing games and worked carefully.

Morphological Classification

students - Inflectional
playing - Inflectional
games - Inflectional
worked - Inflectional
carefully - Derivational
>>>
```

Question 5: Porter Stemming Algorithm 4 –

Analyze | 10 Marks Implement the Porter Stemming Algorithm using Python to identify the root forms of English words.

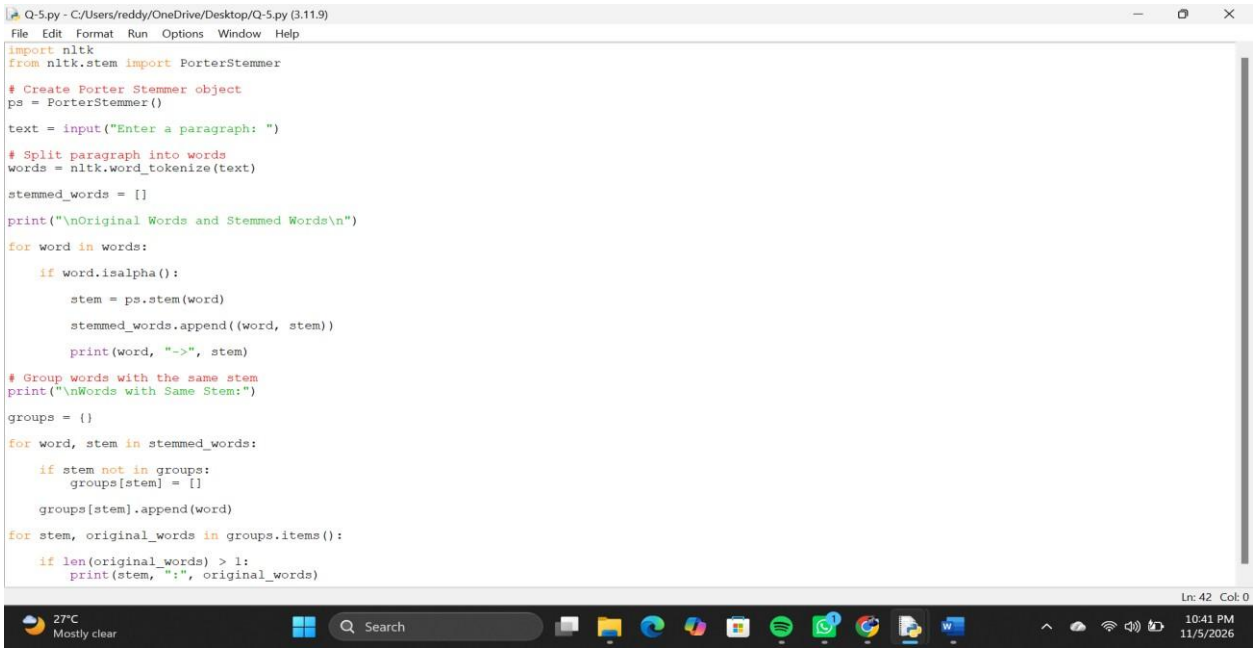
1. Apply stemming to a given paragraph.
2. Display the original words and their stemmed forms.
3. Identify words that produce the same stem.
4. Compare the original and stemmed words.

BTL

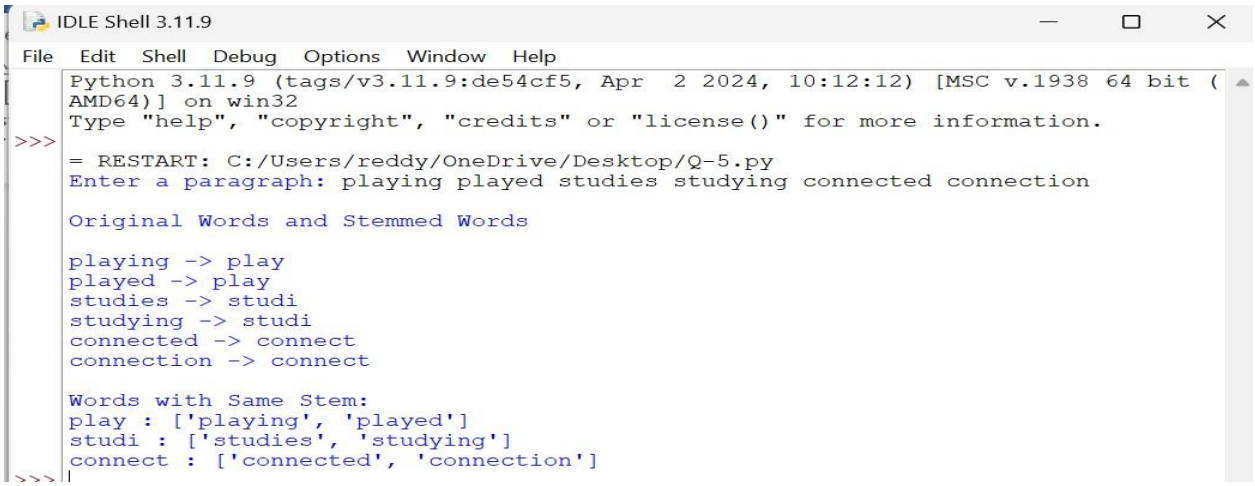
5. Analyze how stemming reduces different word forms to a common base.

Example words: playing, played, studies, studying, connected, connection.

CODE:



OUTPUT:

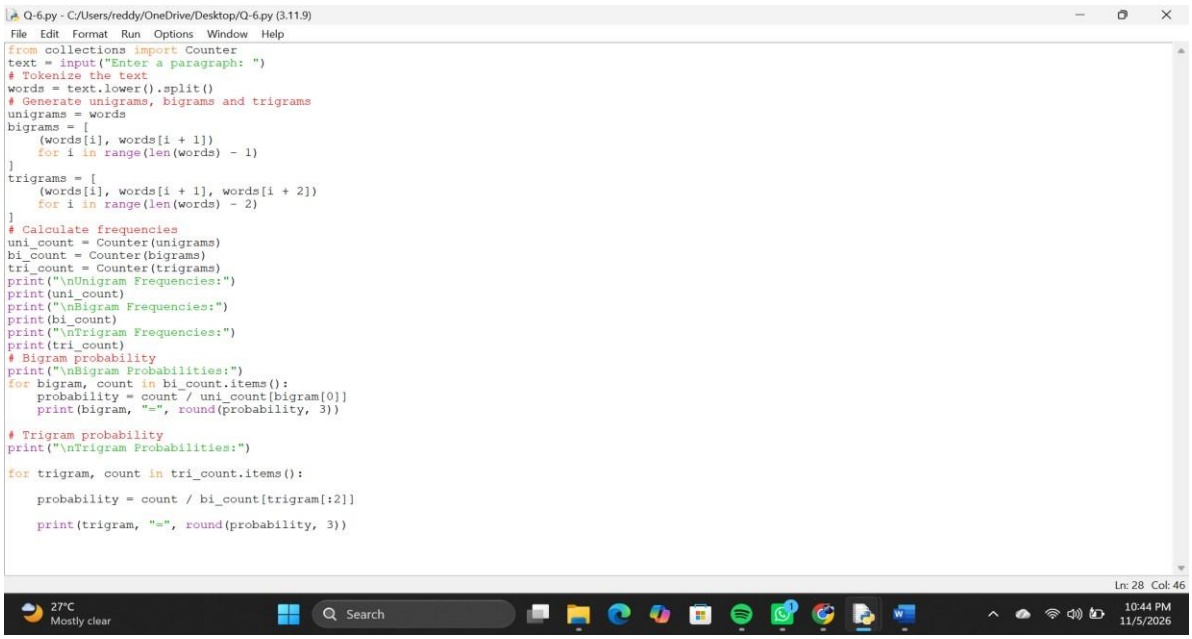


Question 6: Implementation of N-gram Language Models BTL 3 Apply | 10 Marks

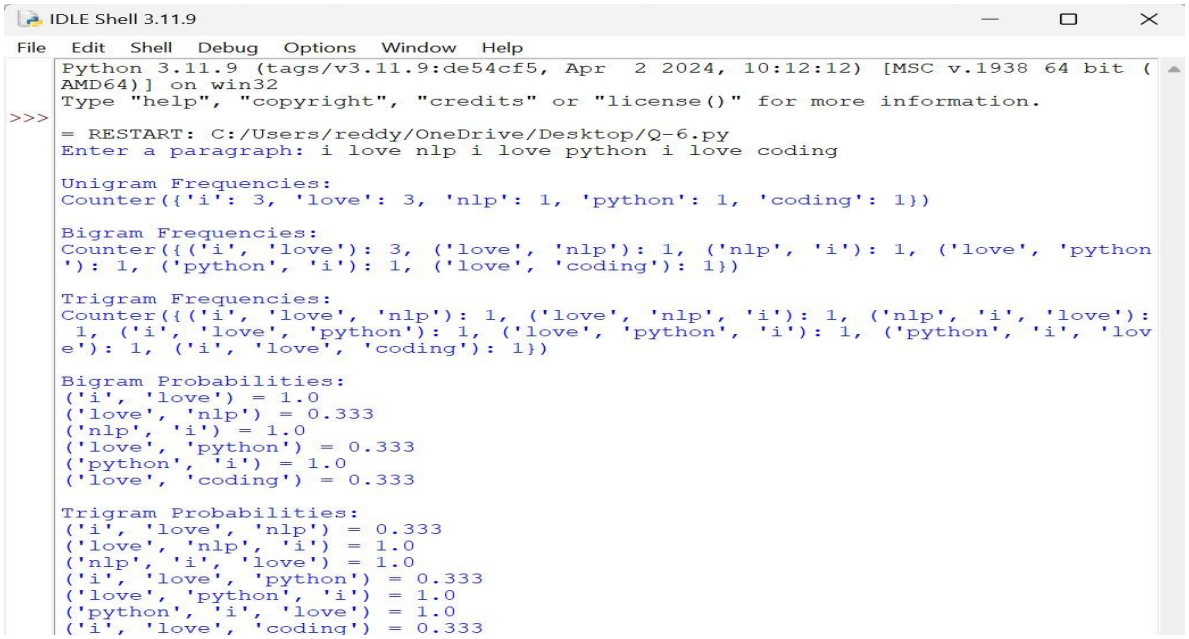
Develop a Python program to implement unigram, bigram and trigram language models for a given English paragraph.

1. Tokenize the given paragraph into individual words.
2. Generate unigram, bigram and trigram sequences.
3. Calculate the frequency of each N-gram.
4. Calculate the probability of selected bigrams and trigrams.
5. Compare the generated N-gram models and display the results.

CODE:



OUTPUT:



Question 7: Word Frequency and N-gram Smoothing BTL 3 Apply | 10 Marks

Write a Python program to calculate word frequencies and implement smoothing techniques for an N-gram language model.

1. Count the occurrences of individual words in a given corpus.
2. Generate bigram frequencies.
3. Calculate unsmoothed bigram probabilities.
4. Implement Add-One (Laplace) smoothing to handle unseen bigrams.
5. Compare the probabilities obtained before and after smoothing.

CODE:

```
*Q-7.py - C:/Users/reddy/OneDrive/Desktop/Q-7.py (3.11.9)*
File  Edit  Format  Run  Options  Window  Help
from collections import Counter
text = input("Enter a paragraph: ")
words = text.lower().split()
# Word frequency
word_count = Counter(words)
# Bigram frequency
bigrams = [
    (words[i], words[i + 1])
    for i in range(len(words) - 1)
]
bigram_count = Counter(bigrams)
vocabulary_size = len(word_count)
total_bigrams = len(bigrams)
print("\nWord Frequencies:")
print(word_count)
print("\nBigram Frequencies:")
print(bigram_count)
print("\nBigram Probabilities:")
for bigram, count in bigram_count.items():
    probability = count / word_count[bigram[0]]
    print(bigram, "=", round(probability, 3))
# Add-One Smoothing
print("\nAdd-One Smoothing:")
test_bigrams = [
    ("i", "love"),
    ("i", "coding")
]
for bigram in test_bigrams:

    count = bigram_count[bigram]

    probability = (
        count + 1
    ) / (
        word_count[bigram[0]] + vocabulary_size
    )

    print(bigram, "=", round(probability, 3))
```

Ln: 27 Col: 1

27°C Mostly clear Search 10:47 PM 11/5/2026

OUTPUT:

```
IDLE Shell 3.11.9
File  Edit  Shell  Debug  Options  Window  Help
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr  2 2024, 10:12:12) [MSC v.1938 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/reddy/OneDrive/Desktop/Q-7.py
Enter a paragraph: i love nlp i love python i love coding

Word Frequencies:
Counter({'i': 3, 'love': 3, 'nlp': 1, 'python': 1, 'coding': 1})

Bigram Frequencies:
Counter({('i', 'love'): 3, ('love', 'nlp'): 1, ('nlp', 'i'): 1, ('love', 'python
'): 1, ('python', 'i'): 1, ('love', 'coding'): 1})

Bigram Probabilities:
('i', 'love') = 1.0
('love', 'nlp') = 0.333
('nlp', 'i') = 1.0
('love', 'python') = 0.333
('python', 'i') = 1.0
('love', 'coding') = 0.333

Add-One Smoothing:
('i', 'love') = 0.5
('i', 'coding') = 0.125
>>>
```

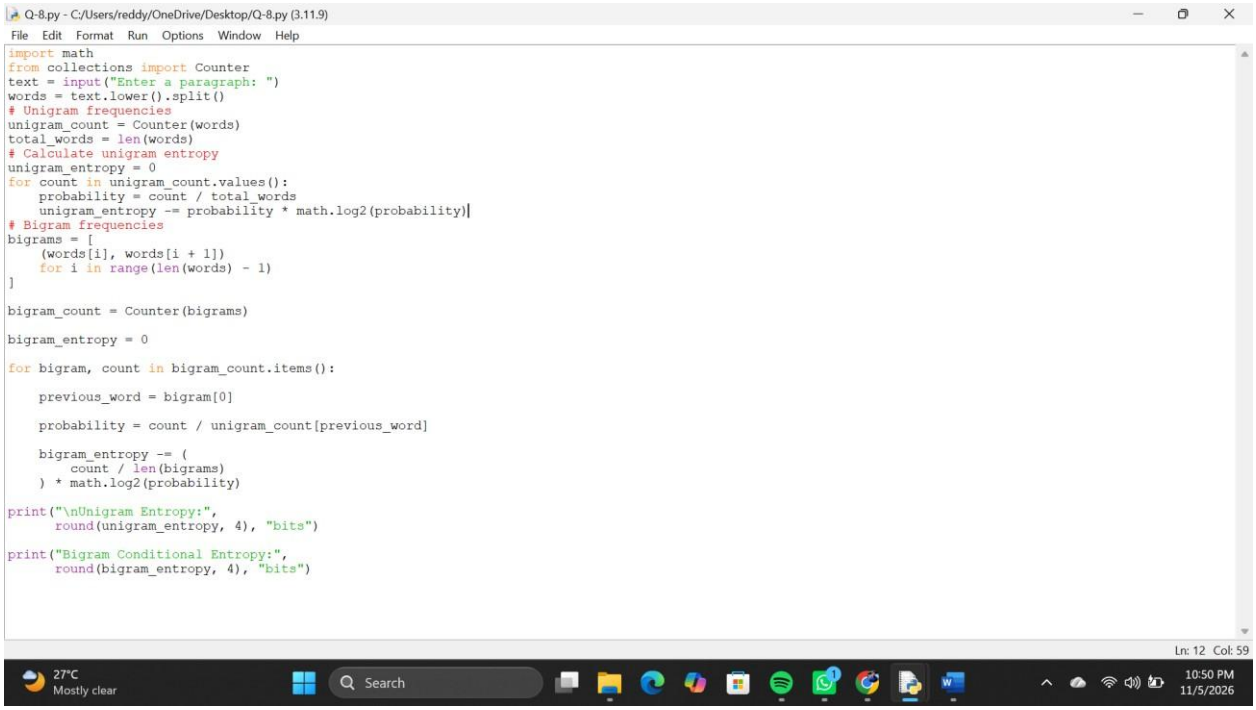
Question 8: Entropy Calculation for Language Models BTL 4

Analyze | 10 Marks Develop a Python program to calculate the entropy of a given text corpus using unigram and bigram probability distributions.

1. Tokenize the given corpus.
2. Calculate unigram and bigram probabilities.
3. Compute entropy using Shannon's entropy formula.
4. Compare the entropy values of unigram and bigram models.

5. Analyze how the context of preceding words influences the uncertainty of the next word.

CODE:



OUTPUT:

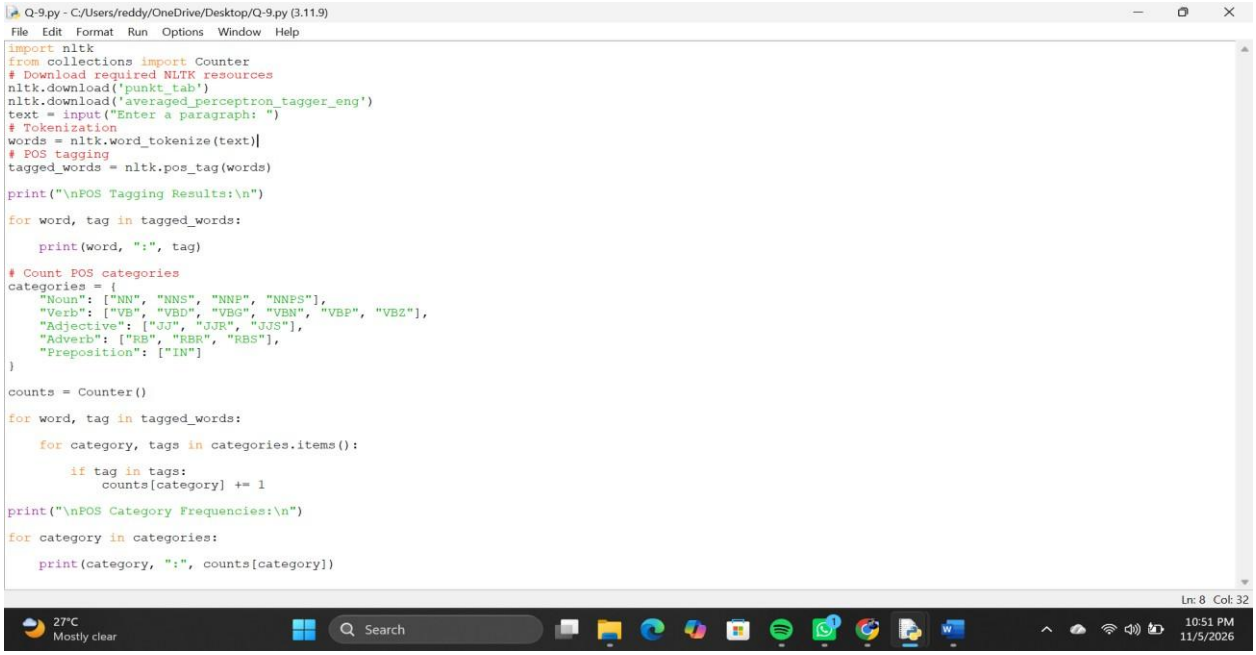


Assignment 9: English Part-of-Speech Tagging BTL 3 – Apply | 10 Marks Develop a Python program to perform Part-of-Speech (POS) tagging on a given English paragraph.

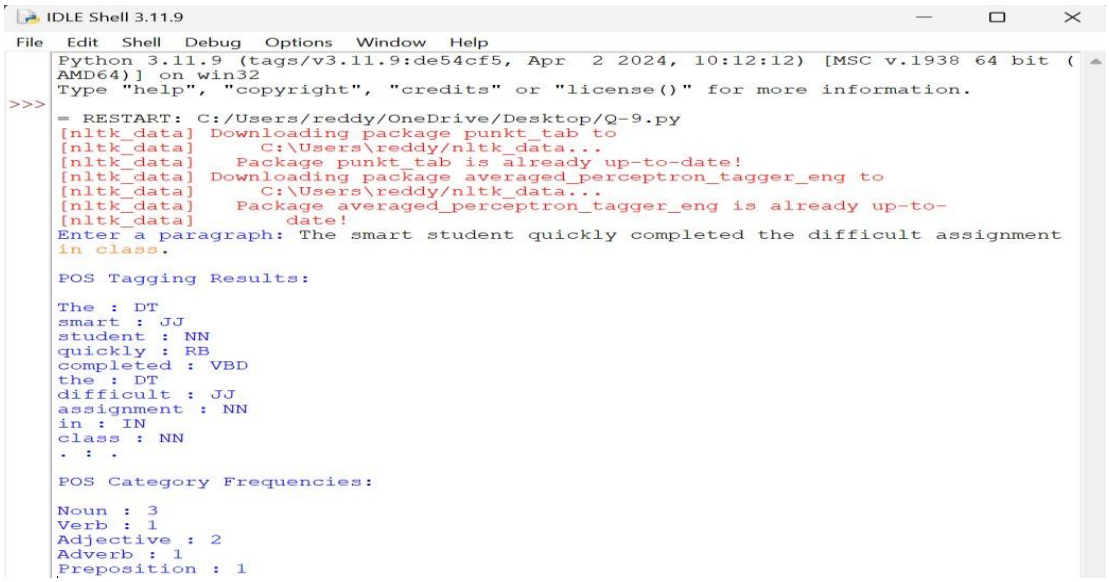
1. Tokenize the input paragraph into words.
2. Identify nouns, verbs, adjectives, adverbs and prepositions.
3. Apply an appropriate English POS tagset.
4. Display each word along with its corresponding POS tag.
5. Count and display the frequency of each POS category.

Suggested Python library: NLTK.

CODE:



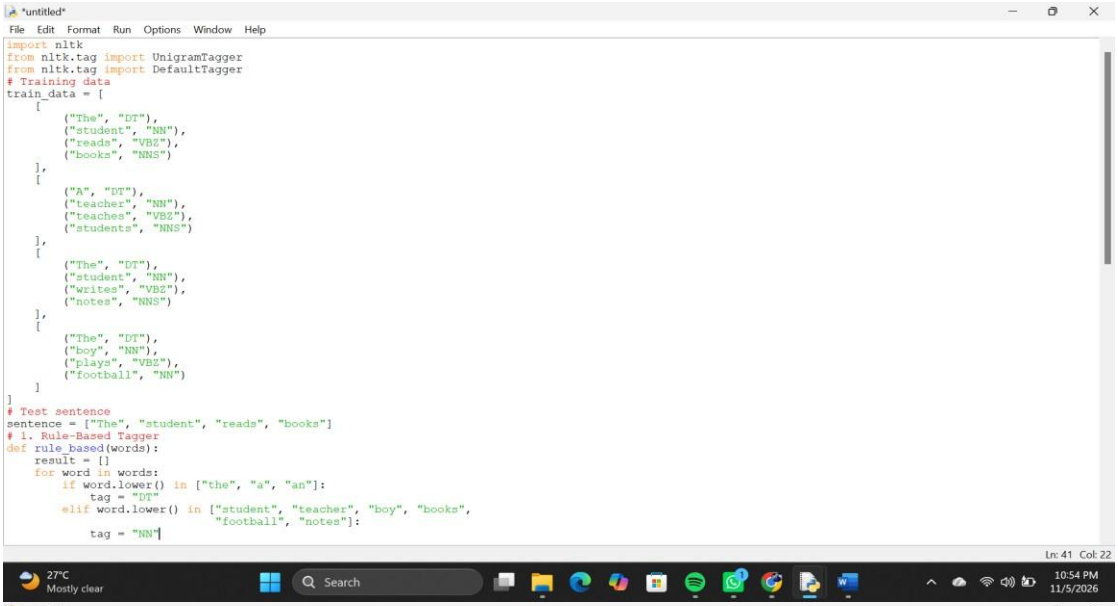
OUTPUT:

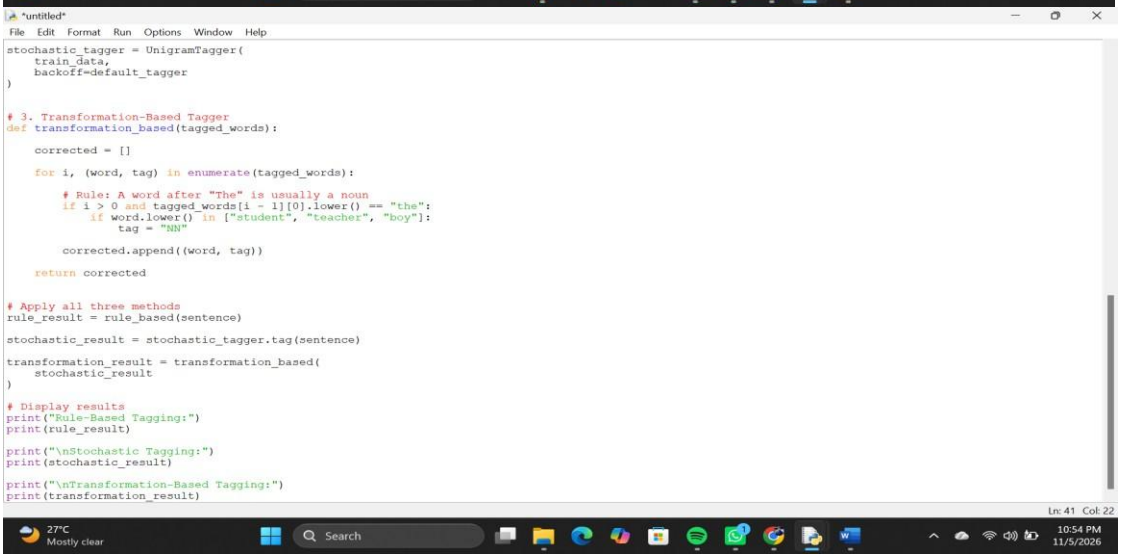


Assignment 10: Comparative Analysis of POS Tagging Technique BTL 4 – Analyze | 10 Marks Develop a Python application to analyze and compare Rule-Based, Stochastic and Transformation-Based POS tagging techniques.

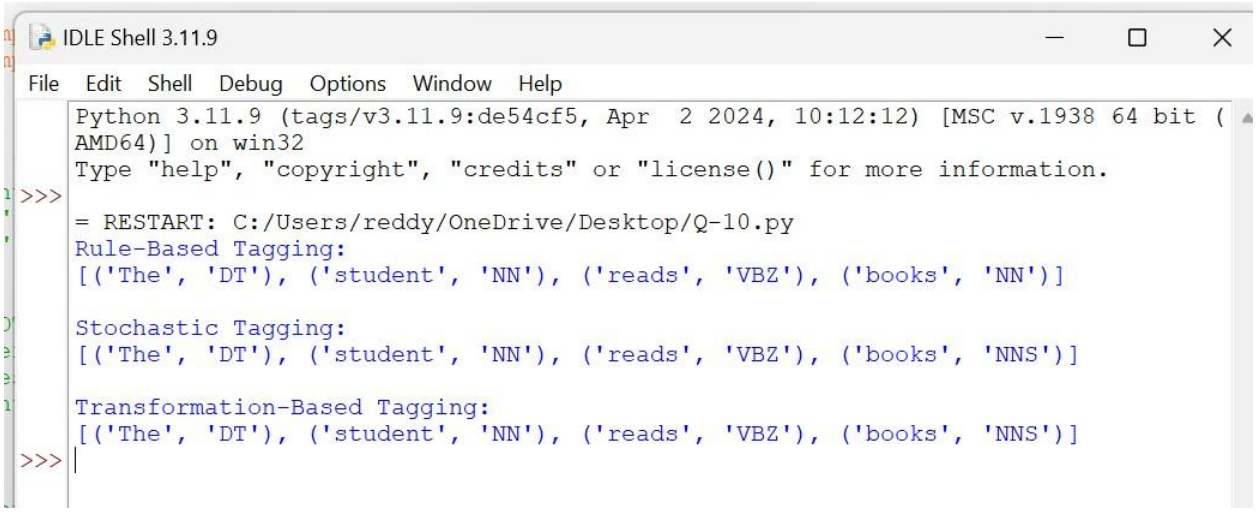
1. Implement a rule-based POS tagger using predefined grammatical rules.
2. Apply a stochastic POS tagger using a trained probabilistic model.
3. Implement or demonstrate transformation-based tagging using an available Python NLP library or a simplified transformation-rule approach.
4. Compare the tagging results for the same set of sentences.
5. Analyze the differences in tagging accuracy and identify cases where the tagging methods produce different results.Suggested Python libraries: NLTK and spaCy.

CODE:


untitled
File Edit Format Run Options Window Help
import nltk
from nltk.tag import UnigramTagger
from nltk.tag import DefaultTagger
Training data
train_data = [
[
("The", "DT"),
("student", "NN"),
("reads", "VBZ"),
("books", "NNS")
],
[
("A", "DT"),
("teacher", "NN"),
("teaches", "VBZ"),
("students", "NNS")
],
[
("The", "DT"),
("student", "NN"),
("writes", "VBZ"),
("notes", "NNS")
],
[
("The", "DT"),
("boy", "NN"),
("plays", "VBZ"),
("football", "NN")
]
]
Test sentence
sentence = ["The", "student", "reads", "books"]
1. Rule-Based Tagger
def rule_based(words):
result = []
for word in words:
if word.lower() in ["the", "a", "an"]:
tag = "DT"
elif word.lower() in ["student", "teacher", "boy", "books",
"football", "notes"]:
tag = "NN"
Ln: 41 Col: 22
27°C
Mostly clear
Search
10:54 PM
11/5/2026


untitled
File Edit Format Run Options Window Help
stochastic_tagger = UnigramTagger(
train_data,
backoff=default_tagger
)
3. Transformation-Based Tagger
def transformation_based(tagged_words):
corrected = []
for i, (word, tag) in enumerate(tagged_words):
Rule: A word after "The" is usually a noun
if i > 0 and tagged_words[i - 1][0].lower() == "the":
if word.lower() in ["student", "teacher", "boy"]:
tag = "NN"
corrected.append((word, tag))
return corrected
Apply all three methods
rule_result = rule_based(sentence)
stochastic_result = stochastic_tagger.tag(sentence)
transformation_result = transformation_based(
stochastic_result
)
Display results
print("Rule-Based Tagging:")
print(rule_result)
print("\nStochastic Tagging:")
print(stochastic_result)
print("\nTransformation-Based Tagging:")
print(transformation_result)
Ln: 41 Col: 22
27°C
Mostly clear
Search
10:54 PM
11/5/2026

OUTPUT:


IDLE Shell 3.11.9
File Edit Shell Debug Options Window Help
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/reddy/OneDrive/Desktop/Q-10.py
Rule-Based Tagging:
[('The', 'DT'), ('student', 'NN'), ('reads', 'VBZ'), ('books', 'NN')]
Stochastic Tagging:
[('The', 'DT'), ('student', 'NN'), ('reads', 'VBZ'), ('books', 'NNS')]
Transformation-Based Tagging:
[('The', 'DT'), ('student', 'NN'), ('reads', 'VBZ'), ('books', 'NNS')]
>>>

Question	Major Concept	BTL Level	Marks
1	Regular Expressions	3	10
2	Finite State Automata	3	10
3	Morphological Analysis	4	10
4	Inflectional and Derivational Morphology	3	10
5	Porter Stemmer	4	10
6	N-gram Models	3	10
7	Word Counting and Smoothing	3	10
8	Entropy	4	10
9	POS Tagging	3	10
10	POS Tagging Techniques	4	10

Evaluation Rubric (10 Marks per Assignment)

Evaluation Criteria	**Marks**
Understanding of the NLP concept	2
Python implementation and logic	3
Correctness of output	2
Analysis and interpretation of results	2
Code readability and documentation	1
Total	10